Desarrollo Backend II (PBY2202)

Evaluación Sumativa 1 – Semana 3: Integrando Seguridad en Aplicaciones Backend

Nombre estudiante:	Víctor Venegas Lindzay
Asignatura:	Desarrollo Backend II (PBY2202)
Carrera / Fecha:	Analista Programador Computacional | 08 Noviembre 2026

1. Descripción de la Actividad

En esta tercera semana, se desarrolla la actividad sumativa 'Integrando seguridad en aplicaciones Backend', donde a partir de una aplicación base proporcionada, se analiza, configura, desarrolla e integran mecanismos de seguridad utilizando frameworks y estándares de la industria como Spring Security 6 y JWT (JSON Web Token). El objetivo es blindar la aplicación frente a amenazas críticas (OWASP Top 10), garantizando interoperabilidad, escalabilidad sin estado y pleno cumplimiento con la normativa chilena sobre protección de datos personales.

Contexto del Caso: Clínica PetSalud

La industria veterinaria en Chile ha jugado un papel clave en el bienestar animal desde mediados del siglo XX. 'CLÍNICA PETSALUD' es una institución de referencia fundada en Santiago en los años 80. Con más de cuatro décadas de trayectoria, cuenta con cuatro sucursales en la Región Metropolitana y la Región de Valparaíso, con planes de expansión hacia otras regiones del país.

PetSalud ofrece atención médica veterinaria especializada, cirugías, planes de vacunación, venta de alimentos balanceados, farmacia y artículos de confort, además de peluquería y programas de entrenamiento. Para modernizar sus operaciones, requiere asegurar un backend desacoplado que gestione expedientes médicos digitales, inventario en tiempo real, ventas y fidelización con altos estándares de seguridad y confidencialidad.

2. Respuestas a las Preguntas de Apoyo

Pregunta 1: Principales amenazas de seguridad que enfrenta 'CLÍNICA PETSALUD' y su impacto en el negocio

Debido a la naturaleza mixta de su negocio (atención clínica presencial, fichas médicas reservadas, comercio electrónico y acumulación de puntos), PetSalud enfrenta amenazas de alto impacto:

- **Falla en el Control de Acceso a Nivel de Objetos (BOLA / IDOR):** Manipulación de identificadores en la URL (/api/expedientes/{id}, /api/citas/{id}) para acceder a fichas médicas de otros dueños de mascotas. Impacto: vulneración de confidencialidad, sanciones legales (Ley 19.628) y pérdida de confianza de los clientes.
- **Inyección SQL (SQL Injection):** Inserción de sentencias SQL en formularios de autenticación o filtros de catálogo (/api/productos/categoria). Impacto: bypass de autenticación, exfiltración de bases de datos de clientes y destrucción o manipulación de inventario de medicamentos.
- **Cross-Site Scripting (XSS):** Inyección de scripts maliciosos en campos abiertos (diagnósticos, motivos de consulta, nombres de productos). Impacto: ejecución de código en los navegadores de veterinarios o administradores, robo de credenciales y secuestro de sesiones.
- **Fallas de Autenticación y Fuerza Bruta:** Contraseñas débiles en texto claro o tokens previsibles. Impacto: suplantación de personal veterinario y alteración de recetas médicas.
- **Cross-Site Request Forgery (CSRF):** Uso de sesiones automáticas por cookies que permitan a sitios externos engañar al navegador para ejecutar transacciones o cancelaciones de citas en nombre del usuario.

Pregunta 2: Estrategias de seguridad más apropiadas en el contexto de comercio electrónico y clínica veterinaria

Las estrategias más adecuadas adoptadas en la solución comprenden:
1. Arquitectura Stateless con JWT: Eliminación de cookies automáticas JSESSIONID; la autenticación exige la cabecera 'Authorization: Bearer <token>', lo que neutraliza por diseño los ataques CSRF cross-origin.
2. Cifrado Adaptativo de Contraseñas (BCrypt): Aplicación de factor de costo 12 (4096 rondas de hashing salado) para impedir la recuperación de claves ante filtraciones.
3. Control de Acceso Basado en Roles (RBAC): Separación estricta de privilegios (CLIENTE, EMPLEADO, GERENTE) mediante reglas HTTP y anotaciones @PreAuthorize.
4. Persistencia Parametrizada (Anti-SQLi): Spring Data JPA y Hibernate aseguran que todas las entradas se procesen como literales en PreparedStatements.
5. Serialización JSON Segura y Cabeceras HTTP: Respuestas exclusivamente bajo Content-Type 'application/json;charset=UTF-8' combinadas con 'X-Content-Type-Options: nosniff'.

Pregunta 3: Framework de seguridad seleccionado y justificación técnica

Se seleccionó Spring Security 6 (integrado en Spring Boot 3.3.6). Sus fortalezas distintivas son:

- Integración nativa no intrusiva con Spring Boot y Spring Data JPA mediante cadenas de filtros (SecurityFilterChain).
- Soporte robusto para arquitecturas Stateless mediante SessionCreationPolicy.STATELESS, permitiendo escalar réplicas del backend en la nube sin sincronizar memoria de servidor.
- Capacidad declarativa avanzada mediante expresiones SpEL (@PreAuthorize).
- Estandarización del manejo de excepciones (AuthenticationEntryPoint y AccessDeniedHandler) para devolver respuestas JSON estructuradas.

Pregunta 4: Implementación de la autenticación y autorización para los roles en Spring Security

La implementación se estructuró a través de cinco componentes coordinados:

1. UsuarioDetailsService: Consulta UsuarioRepository y carga los detalles de la cuenta desde la base de datos.

2. UsuarioDetails: Implementa UserDetails y mapea los roles (CLIENTE, EMPLEADO, GERENTE) tanto con el prefijo canónico 'ROLE_' como con el nombre simple, asegurando total compatibilidad.

3. JwtTokenProvider: Genera y valida tokens firmados con HMAC-SHA256 (clave >= 256 bits), transportando la identidad, ID de usuario y roles con caducidad en 24 horas.

4. JwtAuthenticationFilter: Intercepta cada petición entrante (OncePerRequestFilter), extrae el token Bearer, verifica su firma criptográfica y registra la autenticación en el SecurityContextHolder.

5. SecurityConfig: Ensambla la cadena de filtros, desactiva CSRF justificadamente, protege las rutas por rol y delega el cifrado a BCrypt.

Pregunta 5: Medidas adicionales para el cumplimiento de la Ley de Protección de Datos Personales en Chile

De conformidad con la Ley N° 19.628 sobre Protección de la Vida Privada y el Art. 19 N° 4 de la Constitución Política de la República:

- **Confidencialidad de Expedientes Médicos:** Los diagnósticos y recetas en /api/expedientes son accesibles únicamente para el personal médico asignado y los dueños de cada mascota.
- **No Exposición de Contraseñas:** En la entidad Usuario, el campo password se configuró con @JsonProperty(access = WRITE_ONLY) y @ToString.Exclude, garantizando que jamás se serialicen hashes hacia los clientes ni se registren en logs.
- **Cifrado Fuerte Obligatorio:** Ninguna contraseña se persiste en texto plano.
- **Respuestas de Error Limpias:** Los manejadores de error 401 y 403 devuelven mensajes JSON genéricos sin exponer trazas internas ni detalles del esquema de base de datos.

3. Entregables Oficiales

Entregable 1: Análisis y selección de la estrategia y framework de seguridad

1.1 Análisis Contextualizado y Perfiles de Usuario

El modelo operacional de Clínica PetSalud requiere articular tres estamentos de usuarios con requerimientos y criticidades distintas:

- **Cliente (Dueño de Mascota):** Perfil público autenticado. Puede consultar el catálogo de productos, agendar consultas para sus animales, revisar las citas médicas activas de sus mascotas, realizar compras de alimentos o accesorios y monitorear sus puntos de fidelización. Tiene prohibido el acceso a historiales de terceros o a modificaciones médicas.
- **Empleado (Veterinarios y Asistentes Clínicos):** Perfil técnico operativo. Gestiona la agenda clínica, actualiza estados de citas médicas (atendida, confirmada, cancelada), registra diagnósticos y tratamientos en los expedientes de salud de las mascotas y monitorea alertas de stock crítico en bodega. No tiene privilegios de borrado de fichas ni de visualización de métricas financieras globales.
- **Gerente (Alta Dirección):** Máximo nivel de privilegios. Supervisa las ventas y facturación consolidada de todas las sucursales (Santiago y Valparaíso), gestiona altas y bajas de usuarios y roles, y posee permisos exclusivos de auditoría sobre expedientes clínicos.

1.2 Matriz de Amenazas y Medidas de Mitigación

Amenaza	Escenario PetSalud	Impacto en Negocio	Mitigación Implementada
BOLA / IDOR	Manipulación de IDs en /api/expedientes/{id} o /api/citas/{id}.	Fuga masiva de fichas clínicas e infracción a la Ley 19.628.	Control de acceso granular con @PreAuthorize y validación en servicios.
Inyección SQL	Inyección de sentencias en login o en el filtro de categoría de productos.	Bypass de login, robo masivo de usuarios o alteración de inventario.	Uso de Spring Data JPA con PreparedStatements parametrizados por driver JDBC.
Cross-Site Scripting (XSS)	Inyección de scripts en nombres de mascotas, motivos de cita o productos.	Robo de tokens en navegadores de veterinarios o ejecución de acciones no autorizadas.	Serialización forzada en application/json y cabecera X-Content-Type-Options: nosniff.
CSRF	Peticiones falsificadas desde sitios de terceros para comprar o cancelar citas.	Transacciones no deseadas y desajuste en agendas de los profesionales.	Modelo Stateless: tokens transportados en cabecera Bearer, inmune a envíos automáticos cross-site.
Fuerza Bruta / Robo Claves	Ataques automatizados de diccionario sobre el endpoint de login.	Suplantación de personal médico y alteración de diagnósticos.	Cifrado adaptativo BCrypt (costo 12) y expiración de tokens en 24 horas.

1.3 Cuadro Comparativo de Estrategias de Autenticación

Estrategia	Mecanismo	Ventajas	Desventajas	¿Aplica a PetSalud?
En Memoria	Credenciales hardcodeadas en memoria RAM del servidor.	Configuración rápida para prototipos elementales.	Nula persistencia; no permite registro dinámico ni producción.	No (Solo maquetas)
Base de Datos + Cookies (JDBC)	Usuarios en BD con sesiones de servidor y cookies JSESSIONID.	Persistencia completa y facilidad inicial.	Requiere estado en memoria del servidor; dificulta escalamiento horizontal.	Parcial (Solo para datos)
LDAP / LDAPS	Directorio jerárquico corporativo centralizado (Active Directory).	Excelente para redes corporativas de oficina.	Alta complejidad técnica; no apto para clientes de ecommerce.	No (Sobrecarga)
IDaaS (Auth0 / Okta)	Autenticación delegada en plataforma cloud de terceros.	Soporte MFA y delegación de mantención.	Costos recurrentes por usuario activo y dependencia externa.	Opcional (Fase posterior)
JWT + Spring Security	Tokens criptográficos firmados transmitidos en cabeceras HTTP.	Stateless total, alta escalabilidad, desacople de microservicios y seguro.	Requiere lógica para revocación anticipada de tokens.	**SÍ (Seleccionada)**

1.4 Justificación de la Elección de JWT

La selección de JWT junto a Spring Security 6 se fundamenta en tres pilares estratégicos para Clínica PetSalud:

1. Escalabilidad Horizontal sin Estado (Stateless): Ante los planes de expansión desde Santiago hacia Valparaíso y nuevas regiones, una arquitectura sin estado permite balancear cargas entre múltiples instancias del backend sin sincronizar sesiones en memoria ni requerir servidores de caché como Redis.

2. Autocontenido y Eficiencia: Al incluir los roles y el identificador en los claims verificados mediante HMAC-SHA256, el filtro de autorización resuelve los permisos en cada petición en nanosegundos sin consultar reiterativamente la base de datos.

3. Interoperabilidad Estándar (RFC 7519): Facilita la futura integración con aplicaciones móviles nativas para veterinarios y portales web de clientes sin depender de cookies específicas de navegador.

Entregable 2: Guía de configuración y explicación de la implementación

A continuación se detalla la guía procedimental paso a paso de cómo se configuró y desarrolló la capa de seguridad en Clínica PetSalud, explicando los conceptos, decisiones de diseño y la interconexión entre componentes.

Paso 1: Preparación del Entorno y Configuración de Dependencias (pom.xml)

Para dotar al proyecto de las capacidades de seguridad requeridas bajo Spring Boot 3.3.6 y Java 17/21, se procedió a incorporar las siguientes dependencias clave en el archivo pom.xml:

- **Spring Security Starter:** Activa la infraestructura de filtros de seguridad, el gestor de autenticación y los algoritmos criptográficos.
- **Spring Boot Starter Validation**: Habilita la validación formal de los objetos de transferencia de datos (DTOs) con anotaciones como @NotBlank y @Email, asegurando la integridad de las entradas.
- **Tríada Modular de JJWT (versión 0.12.6):** Se agregaron jjwt-api (interfaces públicas), jjwt-impl (motor criptográfico de firma HMAC) y jjwt-jackson (serialización segura de claims en JSON).
- **Base de Datos Dual:** Se incorporó H2 Database para permitir la ejecución inmediata del proyecto y la evaluación docente en memoria sin dependencias externas, manteniendo mysql-connector-j para ambientes productivos.
- **SpringDoc OpenAPI 2.6.0:** Proporciona la interfaz interactiva de Swagger UI con soporte integrado para pruebas mediante tokens Bearer.

Diagnóstico y Corrección Aplicada: Durante la revisión inicial se constató que el archivo application.properties original provisto contenía una codificación defectuosa (caracteres corruptos en 'tu_contraseña') que impedía la compilación en Maven con un fallo de tipo MalformedInputException. Se normalizó el archivo a UTF-8 y se fijó explícitamente <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> en el pom.xml.

Paso 2: Parametrización de Propiedades de Seguridad (application.properties)

En el archivo de propiedades se configuraron los parámetros del token criptográfico y la persistencia de desarrollo:

- **Clave Secreta (jwt.secret):** Se asignó una clave criptográfica de más de 256 bits de entropía para cumplir con el estándar HMAC-SHA256.
- **Expiración (jwt.expiration):** Se definió un ciclo de vida de 86.400.000 ms (24 horas) para cada token emitido.
- **H2 Database en Memoria:** Se configuró 'jdbc:h2:mem:petsalud_db' con esquema automático (ddl-auto=update) y consola web activa en '/h2-console' para auditoría visual directa de las tablas.

Paso 3: Construcción de la Capa Criptográfica (JwtTokenProvider)

En el paquete security.jwt se implementó la clase JwtTokenProvider, responsable exclusiva del ciclo de vida del token:

- **Inicialización de Clave:** Transforma la cadena secreta de texto en una clave criptográfica real de tipo SecretKey mediante Keys.hmacShaKeyFor().
- **Emisión del Token (generateToken):** Recibe la autenticación del usuario, genera la fecha de emisión y expiración, e incrusta como claims su correo (subject), su ID de base de datos, su nombre completo y su rol formal (ROLE_CLIENTE, ROLE_EMPLEADO, ROLE_GERENTE). Aplica la firma criptográfica con Jwts.SIG.HS256.
- **Validación Robusta (validateToken):** Utiliza el parser criptográfico para comprobar la integridad de la firma y la fecha de caducidad. Si un token ha sido alterado, está malformado o ha expirado, captura ordenadamente las excepciones de JJWT (SignatureException, ExpiredJwtException, etc.) y retorna false sin generar caídas en el servidor.

Paso 4: Creación del Filtro de Intercepción HTTP (JwtAuthenticationFilter)

Para inspeccionar cada solicitud entrante antes de alcanzar los controladores REST, se implementó JwtAuthenticationFilter extendiendo de OncePerRequestFilter:

1. **Intercepción:** Extrae el valor de la cabecera 'Authorization'. Comprueba que inicie con el prefijo 'Bearer '.
2. **Verificación:** Pasa el token al JwtTokenProvider. Si es válido, extrae el correo del usuario.
3. **Reconstrucción de la Identidad:** Invoca a UsuarioDetailsService para recuperar los datos de la cuenta y sus autoridades, construyendo una instancia de UsernamePasswordAuthenticationToken.
4. **Establecimiento del Contexto:** Registra la autenticación en el SecurityContextHolder. Con este paso, el framework reconoce formalmente la identidad y rol del usuario durante el procesamiento de la petición.

Paso 5: Ensamblado de la Configuración Central (SecurityConfig)

Bajo las directrices de Spring Security 6, la clase SecurityConfig define la arquitectura general mediante beans configurados:

- **Sesión Stateless:** Se instruye al servidor a operar mediante SessionCreationPolicy.STATELESS, eliminando la retención de sesiones en memoria y el uso de cookies.
- **Desactivación Justificada de CSRF:** Se deshabilita CSRF debido a que las peticiones se autentican mediante tokens Bearer en cabecera, vector inmune a ataques cruzados automáticos de navegador.
- **Respuestas de Error Tipadas (401 y 403):** Se conectaron dos manejadores dedicados (JwtAuthenticationEntryPoint para accesos anónimos denegados con código 401, y CustomAccessDeniedHandler para accesos con rol insuficiente con código 403), devolviendo payloads JSON estructurados mediante la clase ErrorResponse.
- **Cabeceras Preventivas:** Se configuró frameOptions().sameOrigin() para evitar ataques de clickjacking permitiendo el uso de la consola H2, y contentTypeOptions() contra ataques MIME sniffing.
- **Algoritmo de Hashing:** Se declaró el bean BCryptPasswordEncoder con factor de costo 12.
- **Registro del Filtro:** Se insertó JwtAuthenticationFilter de forma previa a UsernamePasswordAuthenticationFilter.

Paso 6: Gestión de Usuarios, Endpoints de Autenticación y Autorización en Controladores

La integración funcional se completó en las siguientes capas:
- **Entidad y Servicio de Usuarios:** En Usuario.java se marcó el campo password como @JsonProperty(access = WRITE_ONLY) para garantizar que jamás se exponga en respuestas JSON. En UsuarioServiceImpl, todo guardado de clave pasa obligatoriamente por passwordEncoder.encode().
- Controlador de Autenticación (AuthController): Se expusieron los endpoints públicos /api/auth/login (que valida credenciales con el AuthenticationManager y entrega el JWT) y /api/auth/register (que valida datos con RegisterRequest, cifra la clave y registra al cliente).
- **Autorización por Roles en Controladores:** Mediante @EnableMethodSecurity(prePostEnabled = true), se anotaron los controladores del sistema (@PreAuthorize):
  - Rutas Públicas: Catálogo de productos (/api/productos), login, registro, Swagger UI y consola H2.
  - Rol CLIENTE: Agendar y cancelar citas propias (/api/citas), registrar sus mascotas (/api/mascotas), realizar compras (/api/ventas) y consultar sus puntos (/api/recompensas).
  - Rol EMPLEADO: Actualizar estados clínicos de citas (/api/citas/{id}/estado), registrar fichas en expedientes médicos (/api/expedientes) y monitorear productos con stock crítico (/api/productos/stock-bajo).
  - Rol GERENTE: Gestión integral de usuarios (/api/usuarios), auditoría y eliminación de registros clínicos, reportes consolidados de ventas de todas las sucursales y administración de inventario.
- **Semillero de Datos (DataInitializer):** Implementa CommandLineRunner para poblar automáticamente cuentas de prueba cifradas en BCrypt (cliente@petsalud.cl, empleado@petsalud.cl, gerente@petsalud.cl), mascotas e inventario base para pruebas inmediatas.

Entregable 3: Explicación técnica de cómo las configuraciones implementadas protegen los componentes backend frente a las amenazas identificadas

3.1 Protección frente a Inyección SQL (SQL Injection)

Mecanismo Técnico: La aplicación delega el acceso a datos en Spring Data JPA utilizando Hibernate como proveedor de persistencia relacional.

Funcionamiento Interno: Cuando un atacante envía entradas maliciosas (por ejemplo: ' OR '1'='1' -- en el login o sentencias UNION SELECT en el filtro de categoría de productos), Hibernate no concatena directamente la cadena dentro de la instrucción SQL. En su lugar, utiliza consultas preparadas (PreparedStatements) a nivel del driver JDBC. En este modelo, el motor de base de datos compila previamente la estructura de la consulta y asigna las entradas del usuario a marcadores posicionales (?).

Efecto de Protección: Los caracteres especiales como comillas simples, guiones dobles de comentario o comandos SQL son interpretados estrictamente como literales de tipo String y jamás como instrucciones ejecutables. Por consiguiente, la base de datos busca una coincidencia exacta con ese texto malicioso, retornando cero registros sin alterar la lógica de la consulta ni exponer tablas confidenciales.

3.2 Protección frente a Cross-Site Scripting (XSS)

Mecanismo Técnico: Se conjugan el serializador de mensajes Jackson de Spring MVC y las cabeceras HTTP de Spring Security.

Funcionamiento Interno: Todas las respuestas de los controladores se emiten con Content-Type: application/json;charset=UTF-8. Además, Spring Security aplica de forma predeterminada la cabecera 'X-Content-Type-Options: nosniff'.

Efecto de Protección: Si un usuario malintencionado intenta almacenar código ejecutable como <script>alert('XSS')</script> en el nombre de una mascota, motivo de cita o descripción de producto, la información se almacena y viaja como texto plano dentro del objeto JSON. Cuando un navegador consume estos datos, la cabecera 'nosniff' prohíbe terminantemente que el navegador intente interpretar el contenido como HTML ejecutable, neutralizando la posibilidad de que se ejecuten secuencias de comandos en la sesión de los usuarios o veterinarios.

3.3 Protección frente a Cross-Site Request Forgery (CSRF)

Mecanismo Técnico: Desacoplamiento de sesiones y uso exclusivo de Bearer Tokens en cabeceras HTTP (Stateless Architecture).

Funcionamiento Interno: Los ataques de tipo CSRF se basan en que el navegador envía automáticamente las cookies de sesión (JSESSIONID) asociadas a un dominio cada vez que se emite una petición cruzada desde una pestaña o sitio web externo de un atacante.

Efecto de Protección: En nuestro backend, el servidor no gestiona cookies de sesión ni las reconoce para autorizar operaciones. Toda petición autenticada debe incluir explícitamente la cabecera 'Authorization: Bearer <token_jwt>'. Debido a la política del mismo origen (Same-Origin Policy), los navegadores web prohíben que un sitio web de terceros adjunte cabeceras de autorización personalizadas en peticiones cross-site de manera automática. De este modo, cualquier petición falsificada carecerá del token Bearer y será rechazada de inmediato con HTTP 401 Unauthorized.

3.4 Protección frente a Falla de Control de Acceso (BOLA / IDOR)

Mecanismo Técnico: Verificación de identidad centralizada combinando el filtro de seguridad JWT y anotaciones de autorización declarativa @PreAuthorize.

Funcionamiento Interno: En cada petición HTTP, el filtro de seguridad extrae los roles confirmados del token y los almacena en el SecurityContextHolder. Al invocarse un método en un controlador, el interceptor de seguridad evalúa las expresiones de autorización antes de ejecutar la lógica de negocio.

Efecto de Protección: Si un usuario con rol CLIENTE intenta consultar o alterar recursos administrativos (como /api/usuarios para listar cuentas de empleados o /api/citas/{id}/estado para cambiar estados clínicos), la regla de autorización constata la insuficiencia de privilegios e interrumpe de inmediato el procesamiento, respondiendo con un código HTTP 403 Forbidden mediante CustomAccessDeniedHandler.

3.5 Cumplimiento con la Ley 19.628 de Protección de Datos Personales en Chile

Mecanismo Técnico: Cifrado adaptativo irreversible de credenciales, minimización de datos en respuestas JSON y segregación de fichas clínicas.

Funcionamiento Interno: Las contraseñas de los usuarios son procesadas con BCryptPasswordEncoder (12 rondas de hashing salado) y la propiedad password en la entidad Usuario está configurada con @JsonProperty(access = WRITE_ONLY) y @ToString.Exclude.

Efecto de Protección: Este diseño garantiza que ninguna llamada a la API (individual o masiva) filtre hashes de contraseñas hacia los clientes y evita su registro accidental en logs. Del mismo modo, los historiales clínicos veterinarios quedan estrictamente protegidos para consulta exclusiva del personal médico y de los dueños de cada paciente, garantizando la privacidad e integridad de los datos según los mandatos del marco legal chileno.